Documento de Especificação de Requisitos de Software

Sistema Runner

Modelo adaptado conforme ISO/IEC/IEEE 29148:2018

Item	Informação
Nome do sistema	Sistema Runner
Tipo de documento	Especificação de Requisitos de Software (SRS)
Versão do documento	1.0
Contexto	Disciplina de Implementação e Integração de Software — Engenharia de Software
Base documental	especificacao.md, design.md, plano-revisitado-v2.md e sprint-1-tasks.md
Status	Proposta inicial de especificação para desenvolvimento acadêmico

Controle do Documento

Versão	Data	Descrição	Responsável
1.0	28/04/2026	Elaboração inicial da especificação de requisitos do Sistema Runner com base nos documentos fornecidos.	Bárbara Nogueira Carvalho da Silveira

Sumário

1. Introdução
2. Descrição Geral do Sistema
3. Requisitos de Interface Externa
4. Requisitos Funcionais
5. Requisitos Não Funcionais
6. Requisitos de Dados e Persistência Local
7. Restrições de Projeto e Implementação
8. Requisitos de Verificação e Validação
9. Rastreabilidade
10. Apêndices

1. Introdução

1.1 Finalidade do documento

Este documento especifica os requisitos de software do Sistema Runner. O objetivo é descrever, de forma organizada, verificável e rastreável, o que o sistema deve realizar, quais restrições devem ser obedecidas, quais interfaces devem existir e quais critérios serão usados para aceitar a implementação. A estrutura foi elaborada de maneira compatível com a organização típica de uma Especificação de Requisitos de Software prevista pela ISO/IEC/IEEE 29148:2018, sem reproduzir literalmente o texto normativo.

1.2 Escopo do produto

O Sistema Runner é uma solução de linha de comandos destinada a facilitar a execução e a integração de aplicações Java relacionadas à Plataforma HubSaúde. Seu objetivo é permitir que o usuário execute comandos simples para interagir com aplicações Java sem precisar conhecer detalhes de instalação, configuração, invocação por java -jar, controle de processos, download de artefatos ou comunicação HTTP.

O sistema é composto por dois CLIs principais: assinatura, responsável por invocar o assinador.jar para criação e validação simulada de assinaturas digitais; e simulador, responsável por iniciar, parar e consultar o status do Simulador do HubSaúde. O assinador.jar será uma aplicação Java que validará parâmetros e simulará as operações de assinatura e validação.

1.3 Objetivos do sistema

- Permitir a execução de aplicações Java por meio de comandos simples em terminal.
- Ocultar do usuário detalhes técnicos de instalação, configuração e chamada de aplicações Java.
- Oferecer comandos para criação e validação simulada de assinaturas digitais.
- Permitir a execução do assinador.jar em modo local e em modo servidor HTTP.
- Permitir o gerenciamento do ciclo de vida do Simulador do HubSaúde.
- Gerar binários multiplataforma distribuídos por GitHub Releases.

- Incluir mecanismos de verificação de integridade e autenticidade dos artefatos distribuídos.

1.4 Público-alvo do documento

- Estudantes responsáveis pela implementação do Sistema Runner.
- Docentes e avaliadores da disciplina de Implementação e Integração de Software.
- Integradores técnicos interessados no uso do Runner em cenários da Plataforma HubSaúde.
- Desenvolvedores responsáveis por manutenção, testes, integração e documentação do sistema.

1.5 Definições, siglas e abreviações

Termo	Definição
CLI	Interface de Linha de Comandos. Permite que o usuário interaja com o sistema digitando comandos no terminal.
Runner	Sistema responsável por facilitar a execução e integração de aplicações Java por linha de comandos.
assinatura	CLI do Sistema Runner responsável por operações de criação e validação de assinaturas simuladas.
simulador	CLI do Sistema Runner responsável por iniciar, parar e consultar o Simulador do HubSaúde.
assinador.jar	Aplicação Java responsável por validar parâmetros e simular operações de assinatura e validação.
simulador.jar	Aplicação Java/Web do Simulador do HubSaúde, gerenciada pelo CLI simulador.
JDK	Java Development Kit, ambiente necessário para executar e/ou desenvolver aplicações Java.

JRE	Java Runtime Environment, ambiente necessário para executar aplicações Java.
PKCS#11	Interface padrão para comunicação com dispositivos criptográficos, como tokens USB e smart cards.
SemVer	Versionamento semântico no formato MAJOR.MINOR.PATCH, usualmente representado por tags como v0.1.0.
CI/CD	Práticas de Integração Contínua e Entrega/Distribuição Contínua.
Checksum SHA-256	Resumo criptográfico usado para verificar se um arquivo foi alterado.
Cosign/Sigstore	Ferramentas para assinatura e verificação de autenticidade de artefatos de software.

1.6 Referências documentais do projeto

Documento	Uso nesta especificação
especificacao.md	Base principal para visão geral, escopo, requisitos funcionais, integração, entregáveis, simulação, qualidade e assinatura de artefatos.
design.md	Base para atores, sistemas externos, contêineres e protocolos de comunicação.
plano-revisitado-v2.md	Base para planejamento incremental, histórias derivadas, sprints e critérios de aceitação detalhados.
sprint-1-tasks.md	Base para decisões técnicas, layout de pacotes, tarefas da Sprint 1, CI/CD, releases e definição de pronto.

2. Descrição Geral do Sistema

2.1 Perspectiva do produto

O Sistema Runner é um sistema de suporte à execução e integração. Ele não substitui as aplicações Java que executam as regras específicas, mas as encapsula por meio de comandos mais simples. A interação principal ocorre via

terminal. O usuário envia comandos para os CLIs do Runner; os CLIs invocam aplicações Java locais ou serviços HTTP; os resultados são formatados e devolvidos ao terminal.

2.2 Contexto operacional

Elemento	Tipo	Responsabilidade
Usuário	Ator humano	Executa comandos em terminal para criar/validar assinaturas e controlar o simulador.
Sistema Runner	Sistema em desenvolvimento	Fornece CLIs, integra aplicações Java, gerencia processos, downloads e artefatos.
Dispositivo de Assinatura Digital	Sistema externo/hardware	Token ou smart card compatível com PKCS#11 para operações criptográficas.
Simulador do HubSaúde	Sistema externo/aplicação Java/Web	Aplicação gerenciada pelo CLI simulador e acessada por HTTP.

2.3 Componentes principais

Componente	Descrição	Responsabilidades principais
CLI assinatura	Aplicação de linha de comandos em Go.	Receber comandos sign, validate, version, start/stop/status quando aplicável; invocar o assinador.jar; formatar saídas.
assinador.jar	Aplicação Java 21.	Validar parâmetros; simular criação de assinatura; simular validação; expor endpoints HTTP; integrar com PKCS#11 quando aplicável.

CLI simulador	Aplicação de linha de comandos em Go.	Iniciar, parar e consultar o status do simulador.jar; baixar artefatos; verificar portas.
Módulo JDK/JRE	Componente interno do Runner.	Detectar Java instalado; baixar e armazenar ambiente Java compatível quando necessário.
Módulo Release/Download	Componente interno do Runner.	Consultar versões, baixar artefatos e verificar integridade.
Pipeline CI/CD	Automação no GitHub Actions.	Testar, compilar, gerar binários, checksums, assinaturas e releases.

2.4 Funções gerais do produto

- Executar comandos de assinatura e validação simulada por meio do CLI assinatura.
- Invocar o assinador.jar diretamente pelo comando java -jar ou por HTTP quando em modo servidor.
- Formatar os resultados das operações de forma legível no terminal.
- Gerenciar o ciclo de vida do assinador.jar em modo servidor, incluindo início, reutilização, parada e encerramento por inatividade.
- Gerenciar o ciclo de vida do Simulador do HubSaúde, incluindo start, stop e status.
- Baixar JDK/JRE e artefatos Java quando ausentes, respeitando cache local.
- Distribuir CLIs como binários para Windows, Linux e macOS.
- Gerar checksums SHA-256 e assinaturas de artefatos para validação de integridade e autenticidade.

2.5 Classes de usuários

Classe de usuário	Descrição	Necessidades principais
Usuário final técnico	Pessoa que utiliza os CLIs sem querer lidar diretamente com Java.	Comandos simples, mensagens claras,

		instalação mínima, binários prontos.
Integrador da Plataforma HubSaúde	Profissional que precisa executar ou testar aplicações relacionadas ao HubSaúde.	Automação, previsibilidade, controle de processos e integração confiável.
Desenvolvedor do projeto	Pessoa que implementa e mantém o Runner.	Estrutura organizada, testes, CI/CD, rastreabilidade e requisitos verificáveis.
Avaliador da disciplina	Docente ou monitor que avalia a entrega acadêmica.	Documentação clara, critérios de aceitação, rastreabilidade e evidências de funcionamento.

2.6 Ambiente operacional

- Sistemas operacionais alvo: Windows amd64, Linux amd64 e macOS amd64.
- CLIs desenvolvidos em Go 1.25.
- Aplicação assinador.jar desenvolvida em Java 21.
- Uso de terminal/shell como interface principal.
- Uso de diretório local gerenciado, preferencialmente ~/.hubsaude/, para armazenar JDK/JRE, artefatos, cache e registros de processos.
- Uso de GitHub Releases para distribuição de binários e artefatos.

2.7 Premissas e dependências

ID	Premissa/Dependência
PD-01	O usuário terá acesso a um terminal compatível com sua plataforma.
PD-02	O repositório do projeto usará GitHub Actions e GitHub Releases.
PD-03	O ambiente de execução terá acesso à internet quando for necessário baixar JDK/JRE, simulador.jar ou releases.
PD-04	O sistema não realizará assinatura criptográfica real; as operações de

	assinatura e validação serão simuladas.
PD-05	A comunicação com dispositivo criptográfico será feita por interface PKCS#11 quando essa funcionalidade estiver habilitada.
PD-06	O Simulador do HubSaúde expõe endpoints HTTP para consulta de status e encerramento.

3. Requisitos de Interface Externa

3.1 Interface de usuário por linha de comandos

Comando	Finalidade	Observações
assinatura version	Exibir a versão corrente do CLI assinatura.	Deve exibir dev em execução local sem injeção de versão, ou a tag da release em build oficial.
assinatura sign	Solicitar criação de assinatura simulada.	Deve receber parâmetros obrigatórios definidos pelo caso de uso de criação.
assinatura validate	Solicitar validação de assinatura simulada.	Deve receber parâmetros obrigatórios definidos pelo caso de uso de validação.
assinatura stop	Encerrar instância do assinador.jar em execução.	Deve aceitar porta padrão ou parâmetro --port.
simulador start	Iniciar o Simulador do HubSaúde.	Deve verificar porta, localizar/baixar artefatos e iniciar processo.
simulador stop	Parar o Simulador do HubSaúde.	Deve usar o mecanismo HTTP de encerramento definido pelo simulador.
simulador status	Consultar o estado atual do Simulador do HubSaúde.	Deve informar se está em execução e apresentar dados relevantes.

--help	Exibir ajuda de comandos e parâmetros.	Deve existir nos CLIs e subcomandos relevantes.

3.2 Interface entre CLI assinatura e assinador.jar

Modo	Mecanismo	Descrição
Local/CLI	Processo	O CLI executa java -jar assinador.jar com os parâmetros informados pelo usuário.
Servidor HTTP	HTTP	O assinador.jar permanece em execução como servidor e o CLI envia requisições para endpoints de assinatura e validação.

3.3 Interface HTTP do assinador.jar

Endpoint	Método	Finalidade	Resposta esperada
/sign	POST	Criar assinatura simulada a partir de parâmetros válidos.	Objeto estruturado contendo dados da assinatura simulada ou erro.
/validate	POST	Validar assinatura de forma simulada.	Objeto estruturado indicando assinatura válida/inválida ou erro.
/health	GET	Verificar se o servidor está ativo.	Status de saúde simples para reutilização de instância.

3.4 Interface com dispositivo criptográfico

Quando a funcionalidade de material criptográfico estiver habilitada, o assinador.jar deverá utilizar interface compatível com PKCS#11 para comunicação com token ou smart card. Em caso de ausência, indisponibilidade ou erro no dispositivo, a aplicação deverá retornar mensagem clara e não deverá falhar silenciosamente.

3.5 Interface com GitHub Releases e arquivos remotos

O Sistema Runner deverá consultar releases para identificar e baixar artefatos, incluindo binários, simulador.jar, checksums e arquivos de assinatura. Também poderá utilizar um arquivo release.json com URLs e versões para determinar o que precisa ser baixado ou atualizado.

3.6 Interface com sistema de arquivos local

Local/Arquivo	Uso
~/.hubsaude/jdk/ ou equivalente	Armazenar JDK/JRE baixado automaticamente para reutilização.
~/.hubsaude/	Armazenar cache, registros de processo, PID, porta e artefatos baixados.
checksums.txt	Registrar hashes SHA-256 dos binários publicados.
<artefato>.sig e <artefato>.pem	Permitir verificação de autenticidade dos artefatos publicados.

4. Requisitos Funcionais

Os requisitos funcionais abaixo são identificados por RF. Cada requisito possui descrição, racional, critério de verificação e rastreabilidade para as histórias ou decisões do projeto.

RF-001 — Disponibilizar CLI assinatura

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial

Descrição	O sistema deve disponibilizar um executável de linha de comandos denominado assinatura para operações relacionadas à assinatura digital simulada.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar o binário assinatura em Windows, Linux e macOS e confirmar que ele responde a comandos básicos e help.
Origem/rastreabilidade	US-01, US-01.1

RF-002 — Exibir versão do CLI assinatura

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve possuir o comando version, capaz de exibir a versão corrente da aplicação.
Racional	Permite identificar a versão instalada e validar builds locais ou oficiais.
Critério de verificação	Executar assinatura version e confirmar que a saída contém dev em build local ou a tag SemVer em release.
Origem/rastreabilidade	US-01.1; T-01.1.3

RF-003 — Interpretar comando de criação de assinatura

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve aceitar comando para criação de assinatura simulada, com parâmetros obrigatórios definidos pelo caso de uso de criação.
Racional	Necessário para atender ao objetivo do Sistema Runner.

Critério de verificação	Executar assinatura sign com parâmetros válidos e inválidos; validar aceite, rejeição e mensagens de ajuda.
Origem/rastreabilidade	US-01, US-01.2

RF-004 — Interpretar comando de validação de assinatura

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve aceitar comando para validação de assinatura simulada, com parâmetros obrigatórios definidos pelo caso de uso de validação.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar assinatura validate com parâmetros válidos e inválidos; validar aceite, rejeição e mensagens de ajuda.
Origem/rastreabilidade	US-01, US-01.2

RF-005 — Invocar assinador.jar em modo local

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve invocar o assinador.jar diretamente por linha de comandos, usando java -jar e repassando corretamente os parâmetros recebidos.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Teste de integração CLI -> java -jar -> retorno do assinador.jar, validando stdout, stderr e código de saída.
Origem/rastreabilidade	US-01.3

RF-006 — Invocar assinador.jar via HTTP

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve enviar requisições HTTP para o assinador.jar quando este estiver em modo servidor.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Com assinador.jar ativo, executar comandos sign e validate e confirmar que as requisições atingem os endpoints HTTP correspondentes.
Origem/rastreabilidade	US-01, US-01.6

RF-007 — Iniciar assinador.jar como servidor

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve iniciar o assinador.jar em modo servidor usando porta padrão, com possibilidade de porta personalizada por parâmetro.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar comando de inicialização, confirmar processo ativo, porta em escuta, resposta de health check e registro local de PID/porta.
Origem/rastreabilidade	US-01.5

RF-008 — Detectar e reutilizar instância ativa do assinador.jar

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve detectar instância ativa do assinador.jar e

	reutilizá-la, evitando criar processos duplicados.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Iniciar uma instância, executar novo comando e confirmar por health check e PID que a instância existente foi reutilizada.
Origem/rastreabilidade	US-01.7

RF-009 — Parar assinador.jar

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI assinatura deve permitir interromper o assinador.jar em execução na porta padrão ou em porta informada.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar assinatura stop, confirmar encerramento do processo e atualização/remoção do registro local.
Origem/rastreabilidade	US-01.8

RF-010 — Encerrar assinador.jar por inatividade

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O sistema deve permitir configuração de timeout de inatividade para encerramento automático do assinador.jar.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Configurar --timeout, aguardar período sem requisições e confirmar encerramento automático e atualização do registro.
Origem/rastreabilidade	US-01.9

RF-011 — Validar parâmetros no assinador.jar

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O assinador.jar deve validar presença, formato e consistência dos parâmetros recebidos antes de simular criação ou validação de assinatura.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar suíte de testes com parâmetros ausentes, malformados e válidos, confirmando mensagens específicas por parâmetro.
Origem/rastreabilidade	US-02, US-02.2, US-02.3

RF-012 — Criar assinatura simulada

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O assinador.jar deve retornar uma assinatura simulada pré-construída quando receber parâmetros válidos de criação.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Enviar requisição/comando de criação com parâmetros válidos e confirmar resposta estruturada de sucesso com campos previstos.
Origem/rastreabilidade	US-02.1

RF-013 — Validar assinatura simulada

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial

Descrição	O assinador.jar deve retornar resultado simulado de validação, indicando assinatura válida ou inválida conforme critérios simples definidos na implementação.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar validação com entradas de sucesso e falha e confirmar resultado esperado e mensagens claras.
Origem/rastreabilidade	US-02.3

RF-014 — Expor endpoints HTTP do assinador.jar

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O assinador.jar deve expor endpoints HTTP POST /sign e POST /validate, reutilizando a mesma lógica do modo local.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Testes de integração HTTP confirmam status, payloads de sucesso e payloads de erro.
Origem/rastreabilidade	US-02.4

RF-015 — Suportar dispositivo criptográfico via PKCS#11

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O assinador.jar deve suportar interação com dispositivo criptográfico compatível com PKCS#11, quando configurado.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Teste com dispositivo ou simulador equivalente, confirmando sucesso de

	configuração e mensagem clara quando indisponível.
Origem/rastreabilidade	US-02.5

RF-016 — Disponibilizar CLI simulador

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O sistema deve disponibilizar executável de linha de comandos denominado simulador para gerenciamento do Simulador do HubSaúde.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar simulador --help e confirmar comandos start, stop e status.
Origem/rastreabilidade	US-03.3

RF-017 — Iniciar Simulador do HubSaúde

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI simulador deve iniciar o simulador.jar após verificar se a porta necessária está disponível.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar simulador start, confirmar processo ativo, porta 8443 ou configurada em escuta e feedback ao usuário.
Origem/rastreabilidade	US-03, US-03.1

RF-018 — Parar Simulador do HubSaúde

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial

Descrição	O CLI simulador deve encerrar o simulador.jar de forma controlada, usando o mecanismo HTTP previsto pelo simulador.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar simulador stop e confirmar encerramento do processo e indisponibilidade controlada do serviço.
Origem/rastreabilidade	US-03, US-03.2

RF-019 — Consultar status do Simulador do HubSaúde

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI simulador deve consultar e exibir o status atual do simulador.jar, informando se está ou não em execução.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar simulador status com simulador ativo e inativo, validando mensagens de saída.
Origem/rastreabilidade	US-03, US-03.2

RF-020 — Baixar simulador.jar dinamicamente

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI simulador deve obter dinamicamente a versão mais recente do simulador.jar quando ele não estiver disponível localmente ou quando houver versão nova.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Remover cache local, executar start, confirmar download; repetir execução

	e confirmar que o download não é refeito sem mudança de versão.
Origem/rastreabilidade	US-03.4

RF-021 — Permitir URL alternativa de artefato

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O CLI simulador deve aceitar parâmetro --source <url> para indicar fonte alternativa de download do simulador.jar.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar comando com URL alternativa controlada e confirmar download a partir da origem informada.
Origem/rastreabilidade	US-03.4

RF-022 — Detectar JDK/JRE disponível

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O sistema deve detectar se existe Java compatível no PATH ou em diretório local gerenciado.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar em ambiente com e sem Java no PATH e confirmar detecção correta.
Origem/rastreabilidade	US-04, US-04.1

RF-023 — Provisionar JDK/JRE automaticamente

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial

Descrição	Quando Java compatível não estiver disponível, o sistema deve baixar e configurar JDK/JRE adequado à plataforma.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar em ambiente sem Java, confirmar download, armazenamento em ~/.hubsaude/ e uso posterior sem novo download.
Origem/rastreabilidade	US-04, US-04.1

RF-024 — Gerar binários multiplataforma

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O projeto deve gerar binários para Windows amd64, Linux amd64 e macOS amd64.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar pipeline de build e confirmar artefatos para as três plataformas.
Origem/rastreabilidade	US-05, US-05.1; DT-03

RF-025 — Publicar releases versionadas

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	O projeto deve publicar binários em GitHub Releases com versionamento semântico.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Criar tag v0.1.0 ou superior, executar release workflow e confirmar publicação dos binários.
Origem/rastreabilidade	US-05.2

RF-026 — Gerar checksums SHA-256

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	Cada release deve conter checksums SHA-256 dos artefatos distribuídos.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Confirmar presença de checksums.txt e validar hash de cada binário publicado.
Origem/rastreabilidade	US-05, US-05.3; DT-05

RF-027 — Assinar artefatos com Cosign

Campo	Conteúdo
Tipo	Funcional
Prioridade	Essencial
Descrição	Cada artefato distribuído deve possuir arquivos de assinatura e certificado compatíveis com verificação via Cosign.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar cosign verify-blob com .sig, .pem e artefato e confirmar assinatura válida.
Origem/rastreabilidade	US-05.3

5. Requisitos Não Funcionais

Os requisitos não funcionais são identificados por RNF e descrevem atributos de qualidade, restrições e critérios mensuráveis de aceitação.

RNF-001 — Portabilidade multiplataforma

Campo	Conteúdo
Tipo	Não funcional - Portabilidade
Prioridade	Essencial
Descrição	Os CLIs assinatura e simulador devem funcionar em Windows

	amd64, Linux amd64 e macOS amd64.
Racional	O projeto exige distribuição para diferentes sistemas operacionais.
Critério de verificação	Executar testes e/ou binários nas três plataformas alvo por meio do GitHub Actions.
Origem/rastreabilidade	US-05; DT-03

RNF-002 — Usabilidade por terminal

Campo	Conteúdo
Tipo	Não funcional - Usabilidade
Prioridade	Essencial
Descrição	As mensagens de ajuda, sucesso e erro devem ser compreensíveis para usuários sem conhecimento aprofundado de Java ou integração.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Inspecionar --help e saídas de erro; executar testes com entradas inválidas e confirmar indicação clara de correção.
Origem/rastreabilidade	Objetivos específicos; US-01.4

RNF-003 — Confiabilidade no controle de processos

Campo	Conteúdo
Tipo	Não funcional - Confiabilidade
Prioridade	Essencial
Descrição	O sistema deve evitar processos duplicados desnecessários, registrar PID/porta e tratar processos não responsivos.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Testar inicialização repetida, falha de health check e limpeza de registro local.
Origem/rastreabilidade	US-01.7; US-03.2

RNF-004 — Tratamento estruturado de erros

Campo	Conteúdo
Tipo	Não funcional - Confiabilidade/Usabilidade
Prioridade	Essencial
Descrição	Erros devem ser capturados, propagados e apresentados com informação suficiente para correção.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar cenários de JDK ausente, jar ausente, porta ocupada, parâmetro inválido e dispositivo indisponível.
Origem/rastreabilidade	Seção de tratamento de erros; padrões de qualidade

RNF-005 — Desempenho em modo servidor

Campo	Conteúdo
Tipo	Não funcional - Desempenho
Prioridade	Essencial
Descrição	Quando houver instância ativa do assinador.jar em modo servidor, o CLI deve reutilizá-la para evitar o custo de inicialização da JVM a cada operação.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar duas chamadas consecutivas e confirmar que não há novo processo Java quando health check da instância atual é válido.
Origem/rastreabilidade	US-01; modos local e HTTP

RNF-006 — Manutenibilidade da estrutura do projeto

Campo	Conteúdo
Tipo	Não funcional - Manutenibilidade
Prioridade	Essencial

Descrição	O código deve manter separação entre comandos, invocação, detecção/provisionamento de Java e lógica de release/download.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Inspecionar estrutura de diretórios e confirmar aderência ao layout definido.
Origem/rastreabilidade	DT-06

RNF-007 — Testabilidade

Campo	Conteúdo
Tipo	Não funcional - Testabilidade
Prioridade	Essencial
Descrição	O sistema deve possuir testes unitários, de integração e de aceitação para fluxos principais e cenários de erro.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar go test ./..., testes Java e testes de integração definidos na pipeline.
Origem/rastreabilidade	Entregáveis; Sprint 1; critérios das histórias

RNF-008 — Integridade dos artefatos

Campo	Conteúdo
Tipo	Não funcional - Segurança
Prioridade	Essencial
Descrição	Todos os artefatos distribuídos devem permitir verificação por checksum SHA-256.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Comparar hash local de cada binário com checksums.txt publicado na release.
Origem/rastreabilidade	US-05; DT-05

RNF-009 — Autenticidade dos artefatos

Campo	Conteúdo
Tipo	Não funcional - Segurança
Prioridade	Essencial
Descrição	Todos os artefatos distribuídos devem ser assinados com Cosign, incluindo arquivos .sig e .pem.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar verificação de assinatura para cada artefato publicado.
Origem/rastreabilidade	US-05.3; Integridade e assinatura de artefatos

RNF-010 — Não persistência de assinaturas

Campo	Conteúdo
Tipo	Não funcional - Segurança/Escopo
Prioridade	Essencial
Descrição	O sistema não deve implementar armazenamento persistente de assinaturas digitais como funcionalidade de negócio.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Inspecionar código e documentação; confirmar ausência de banco de dados ou repositório persistente de assinaturas.
Origem/rastreabilidade	Itens fora do escopo

RNF-011 — Compatibilidade com Java 21

Campo	Conteúdo
Tipo	Não funcional - Compatibilidade
Prioridade	Essencial
Descrição	O assinador.jar deve ser desenvolvido e executado em ambiente Java 21 compatível.

Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Compilar e executar o projeto Java com Java 21 nos ambientes de CI ou desenvolvimento.
Origem/rastreabilidade	Premissas do plano revisado

RNF-012 — Compatibilidade com Go 1.25

Campo	Conteúdo
Tipo	Não funcional - Compatibilidade
Prioridade	Essencial
Descrição	Os CLIs devem ser desenvolvidos e compilados com Go 1.25.
Racional	Necessário para atender ao objetivo do Sistema Runner.
Critério de verificação	Executar go version na pipeline e confirmar setup-go configurado para versão 1.25.
Origem/rastreabilidade	Premissas; T-01.1.0

6. Requisitos de Dados e Persistência Local

6.1 Dados de configuração local

ID	Dado	Descrição	Persistência
RD-001	Diretório gerenciado	Diretório local usado para armazenar JDK/JRE, artefatos baixados, cache e registros de processo.	~/.hubsaude/ ou equivalente por plataforma
RD-002	Registro de processo	Informações de PID, porta, modo de execução e timestamp de última interação para assinador.jar e simulador.jar.	Arquivo local em formato estruturado, por exemplo JSON

RD-003	Cache de artefatos	Arquivos baixados, versões e metadados para evitar downloads repetidos.	Diretório local gerenciado
RD-004	release.json	Arquivo remoto com versão e URL dos artefatos Java/JRE.	Origem remota e cópia local opcional
RD-005	checksums.txt	Lista de hashes SHA-256 dos binários publicados.	GitHub Releases e cópia local quando necessário

6.2 Estrutura conceitual do release.json

O arquivo de versões poderá conter URLs e versões dos artefatos necessários. Exemplo conceitual:

```
{
  "jar": { "url": "https://.../simulador.jar", "version": "1.2.0" },
  "jre": {
    "windows_x64": "https://...",
    "linux_x64": "https://...",
    "mac_x64": "https://..."
  }
}
```

6.3 Saída conceitual de assinatura simulada

Campo	Descrição	Obrigatoriedade
status	Indica sucesso ou erro da operação.	Obrigatório
signatureId	Identificador da assinatura simulada.	Obrigatório em sucesso
algorithm	Algoritmo informado ou simulado para composição da resposta.	Quando aplicável

signedAt	Data/hora simulada ou real da geração da resposta.	Obrigatório em sucesso
message	Mensagem legível para o usuário.	Obrigatório
errors	Lista de erros de validação quando houver falha.	Obrigatório em erro

6.4 Saída conceitual de validação simulada

Campo	Descrição	Obrigatoriedade
status	Indica sucesso ou erro da operação de validação.	Obrigatório
valid	Valor booleano indicando assinatura válida ou inválida no cenário simulado.	Obrigatório em sucesso
reason	Motivo do resultado, principalmente quando inválido.	Quando aplicável
validatedAt	Data/hora da validação.	Obrigatório em sucesso
errors	Lista de erros de validação de parâmetros.	Obrigatório em erro

7. Restrições de Projeto e Implementação

ID	Restrição
RP-001	Os CLIs devem ser desenvolvidos em Go 1.25.
RP-002	O assinador.jar deve ser desenvolvido em Java 21.
RP-003	O repositório deve usar branch principal main.
RP-004	O módulo Go deve seguir a decisão técnica github.com/kyriosdata/runner, salvo alteração formal do projeto.
RP-005	Os artefatos devem seguir convenção de nome

	assinatura-<versão>-<os>-<arch> e equivalente para simulador quando aplicável.
RP-006	A distribuição deve ocorrer via GitHub Releases.
RP-007	O sistema não deve implementar assinatura digital criptográfica real, validação criptográfica real, autenticação de usuários ou geração de certificados.
RP-008	O projeto deve ser incremental, organizado nas sprints previstas no plano revisado.
RP-009	A interface principal do usuário deve ser linha de comandos, sem interface gráfica.
RP-010	A comunicação com token/smart card deve ocorrer por PKCS#11 quando essa funcionalidade for implementada.

8. Requisitos de Verificação e Validação

8.1 Estratégia de verificação

A verificação deve combinar análise estática, testes automatizados, testes de integração e validação manual dos artefatos publicados. A pipeline de CI/CD deve ser considerada parte da evidência de atendimento dos requisitos.

Tipo de verificação	Aplicação esperada
Compilação local	go build ./..., build Java/Maven e validação de que os binários são gerados sem erro.
Análise estática	go vet ./... e ferramentas equivalentes para Java quando adotadas.
Testes unitários	Validação de funções internas de parsing, validação de parâmetros e respostas simuladas.
Testes de integração	Fluxos CLI -> assinador.jar local, CLI -> HTTP -> assinador.jar, CLI simulador -> simulador.jar.

Testes de aceitação	Execução de comandos finais conforme critérios de aceitação de cada requisito.
Validação de release	Criação de tag SemVer, publicação de artefatos, checksums e assinaturas.

8.2 Critérios gerais de aceite

- Todos os requisitos essenciais devem possuir implementação e teste correspondente.
- Comandos inválidos devem retornar mensagens claras e código de saída adequado.
- Os CLIs devem compilar e executar nas três plataformas-alvo.
- O release deve conter binários, checksums e, quando exigido, arquivos .sig e .pem.
- O modo servidor deve evitar inicializações repetidas desnecessárias do assinador.jar.
- Downloads de JDK/JRE e simulador.jar não devem ser repetidos quando a versão local já estiver atualizada.
- A documentação deve conter exemplos de uso, guia de instalação e explicação dos fluxos principais.

8.3 Definição de pronto da Sprint 1

- go build ./... passa sem erros localmente.
- go vet ./... passa sem warnings relevantes.
- assinatura version exibe a versão correta.
- Workflow de build executa com sucesso em push para main.
- Binários para Windows, Linux e macOS são gerados como artefatos.
- Workflow de release executa ao criar uma tag v*.
- Release v0.1.0 ou equivalente é publicado no GitHub com binários e checksums.txt.

9. Rastreabilidade

9.1 Rastreabilidade entre épicos e requisitos funcionais

Épico/História	Requisitos relacionados

US-01 — Invocar assinador.jar via CLI	RF-001 a RF-010
US-02 — Simular assinatura digital com validação	RF-011 a RF-015
US-03 — Gerenciar ciclo de vida do Simulador	RF-016 a RF-021
US-04 — Provisionar JDK automaticamente	RF-022, RF-023
US-05 — Disponibilizar binários multiplataforma	RF-024 a RF-027

9.2 Rastreabilidade entre sprints e entregas

Sprint	Foco	Requisitos principais
Sprint 1	Fundação e entrega contínua	RF-001, RF-002, RF-024, RF-025, RF-026, RF-027, RNF-001, RNF-006, RNF-007
Sprint 2	Assinatura simulada em modo local	RF-003, RF-004, RF-005, RF-011, RF-012, RF-013, RF-022, RF-023
Sprint 3	Modo servidor e PKCS#11	RF-006, RF-007, RF-008, RF-009, RF-010, RF-014, RF-015
Sprint 4	Simulador e segurança final	RF-016, RF-017, RF-018, RF-019, RF-020, RF-021

9.3 Rastreabilidade entre requisitos e verificação

Grupo	Forma de verificação predominante
RF-001 a RF-004	Testes de comandos CLI, parsing, help e mensagens de erro.
RF-005 a RF-010	Testes de integração com processos Java, HTTP, PID, porta e timeout.
RF-011 a RF-015	Testes unitários e de integração do assinador.jar, incluindo entradas válidas e inválidas.

RF-016 a RF-021	Testes com simulador.jar, endpoints HTTP e cache de artefatos.
RF-022 a RF-023	Testes de ambiente com Java presente e ausente.
RF-024 a RF-027	Validação da pipeline, release, checksums e assinatura Cosign.
RNF-001 a RNF-012	Combinação de CI/CD, inspeção técnica, testes automatizados e validação manual.

10. Apêndices

10.1 Fluxo lógico de criação de assinatura

1. Usuário executa comando de criação no CLI assinatura.
2. O CLI valida a presença mínima de parâmetros e prepara a chamada.
3. O CLI invoca o assinador.jar em modo local ou via HTTP.
4. O assinador.jar valida rigorosamente os parâmetros.
5. O assinador.jar retorna uma assinatura simulada ou mensagem de erro.
6. O CLI formata o resultado e apresenta ao usuário.

10.2 Fluxo lógico de validação de assinatura

1. Usuário executa comando de validação no CLI assinatura.
2. O CLI valida a presença mínima de parâmetros e prepara a chamada.
3. O CLI invoca o assinador.jar em modo local ou via HTTP.
4. O assinador.jar valida os parâmetros recebidos.
5. O assinador.jar retorna resultado simulado de validação ou mensagem de erro.
6. O CLI formata o resultado e apresenta ao usuário.

10.3 Fluxo lógico de gerenciamento do simulador

1. Usuário executa simulador start, simulador stop ou simulador status.
2. O CLI verifica artefatos locais, porta e ambiente Java.
3. Quando necessário, baixa simulador.jar ou JDK/JRE compatível.
4. O CLI inicia, encerra ou consulta o simulador por HTTP.
5. O CLI atualiza registros locais e apresenta feedback ao usuário.

10.4 Itens explicitamente fora do escopo

- Implementação real de assinatura digital criptográfica.
- Implementação real de validação criptográfica de assinatura digital.
- Integração com autoridades certificadoras.
- Armazenamento persistente de assinaturas.
- Interface gráfica.
- Autenticação de usuários.
- Geração de certificados digitais.

10.5 Observação sobre requisitos ainda dependentes de detalhamento

Alguns parâmetros específicos dos comandos sign e validate dependem do detalhamento final dos casos de uso de criação e validação de assinatura. Assim, esta especificação define a obrigação de validar presença, formato e consistência dos parâmetros, mas recomenda alinhar os nomes finais dos parâmetros às especificações FHIR de segurança citadas no material do projeto antes da implementação definitiva.